

BASH

LINUX - RAN LEVIN

Command	Description	Example
ls -l	Lists files in a single column.	ls -l
ls -l	Lists files in a detailed format with permissions, owner, size, etc.	ls -l
ls -a	Lists all files, including hidden ones.	ls -a
ls -h	Lists files with sizes in a human-readable format.	ls -lh
mkdir -p <dir1/dir2>	Creates a directory and any necessary parent directories.	mkdir -p dir1/dir2
touch <file>	Creates an empty file.	touch file.txt
touch <. <file>< td=""><td>Creates an empty hidden file (due to the dot prefix).</td><td>touch .hiddenfile</td></file><>	Creates an empty hidden file (due to the dot prefix).	touch .hiddenfile
cp <source> <destination>	Copies a file to the specified destination path.	cp /path/to/source/file.txt /path/to/destination/
cp <file> <path>	Copies a file from the current directory to another specified path.	cp file.txt /another/directory/
cp -r <directory> <dst>	Recursively copies a directory and its contents.	cp -r dir1 /path/to/destination/
mv <source> <destination>	Moves a file or directory to a new location or renames it.	mv file.txt /new/path/
mv <old name> <new name>	Renames a file or directory.	mv oldname.txt newname.txt
rm <file>	Removes a file.	rm file.txt
rm -r <directory>	Recursively removes a directory and its contents.	rm -r dir1
nano <file>	Opens a file in the Nano text editor, creating it if it doesn't exist.	nano file.txt
cat <file>	Displays the contents of a file.	cat file.txt
pwd	Displays the full path of the current directory.	pwd
cd -	Changes to the previous directory.	cd -
cd ..	Moves up one directory.	cd ..
cd ~	Changes to the home directory.	cd ~
cd /full/path	Changes to the specified full directory path.	cd /path/to/directory
echo <text>	Prints text to the terminal.	echo "Hello, World!"
mv <src> <dst>	Moves a file or directory to a new location or renames it.	mv file.txt newfile.txt
cut -d " " -f 3 <file>	Cuts the third field from each line using space as the delimiter.	cut -d " " -f 3 file.txt

Operators and streams

Operator/Stream	Description	Example
&&	Logical AND. Executes the next command only if the previous command succeeds. There is a dependency between the commands.	mkdir newdir && cd newdir && touch file.txt
;	Command separator. There is no dependency between the commands.	echo "First command"; echo "Second command"
	Pipe. Passes the standard output (stdout) of one command as the input (stdin) to another command. The first command must succeed for the second to process its output.	cat file.txt grep "pattern"
&	Pipe both standard output and standard error (stderr) to the next command.	command1 & command2
>	Redirects standard output to overwrite a file.	echo "Hello, World!" > file.txt
>>	Redirects standard output to append to a file.	echo "This is appended." >> file.txt
<	Redirects standard input from a file.	sort < unsorted.txt
2>	Redirects standard error to a file, overwriting it.	command 2> error.log
2>>	Redirects standard error to a file, appending to it.	command 2>> error.log
&>	Redirects both standard output and standard error to a file.	command &> output_and_error.log
&>>	Redirects both standard output and standard error to append to a file.	command &>> output_and_error.log
/dev/null	Discards standard output or error. Useful for suppressing unwanted output.	command > /dev/null 2>&1

manipulating text and data

Grep	Description	Example
-i	Ignore case distinctions in the pattern and the input file(s).	grep -i "pattern" file.txt
-e or -E	Use extended regular expressions for more complex search patterns.	`grep -E "pattern1
-c	Count the number of lines that match the search pattern.	grep -c "pattern" file.txt
-o	Display only the matching part of each line, rather than the entire line.	grep -o "pattern" file.txt
-v	Invert the search to display only lines that do not match the search pattern.	grep -v "pattern" file.txt

sort	Description	Example
-n	Sort numerically.	sort -n file.txt
-r	Sort in reverse order.	sort -r file.txt
-u	Remove duplicate lines from the output.	sort -u file.txt
-t <char>	Specifies the field delimiter.	sort -t "," -k 2 file.txt
-k <n>	Sort based on the nth field.	sort -k 3 file.txt

cut	Description	Example
-d <char>	Specifies the delimiter that separates fields in the input.	cut -d ":" -f 1 file.txt
-f <n>	Selects the specified fields.	cut -f 1,3 file.txt
-f <n-m>	Selects the specified range of fields.	cut -f 1-3 file.txt

manipulating text and data

Command	Description	Example
more	View file content one page at a time. Use space to go to the next page, and q to quit.	more file.txt
less	Similar to more, but allows backward navigation and more advanced search features. Use / to search, n and N.	less file.txt

Command	Description	Example
uniq		
-c	Prefix lines by the number of occurrences.	uniq -c file.txt
-d	Only print duplicate lines, one for each group.	uniq -d file.txt
-u	Only print unique lines.	uniq -u file.txt

Command	Description	Example
wc		
-l	Count the number of lines.	wc -l file.txt
-w	Count the number of words.	wc -w file.txt
-c	Count the number of bytes (characters).	wc -c file.txt

Environment Variables

Variable	Description	Example
PATH	A list of directories where the system looks for executable files. Each path is separated by a colon (:).	/usr/local/bin:/usr/bin:/bin
HOME	The current user's home directory.	/home/username
USER	The name of the currently logged-in user.	username
SHELL	The path to the current user's default shell.	/bin/bash
PWD	The current working directory.	/home/username/projects
OLDPWD	The previous working directory.	/home/username
TERM	The type of terminal to emulate when running the shell.	xterm-256color
EDITOR	The default text editor used by commands that require text editing.	vim or nano
HISTFILE	The file where the command history is saved.	/home/username/.bash_history
LOGNAME	The name of the currently logged-in user, typically the same as USER.	username
HOSTNAME	The name of the host system.	mycomputer.local
PS1	The primary prompt string, which defines the command prompt appearance.	\u@\h:\w\\$_
CPU	Information about the CPU architecture.	x86_64 or arm64
OSTYPE	The type of operating system.	linux-gnu

Globber patterns

Pattern	Description	Example
*	Matches any string, including an empty string.	<code>ls *.txt</code> - Matches all files ending with .txt
?	Matches any single character.	<code>ls file?.txt</code> - Matches files like file1.txt, fileA.txt, but not file10.txt
[-]	Matches any single character from the specified set. Can also specify a range of characters.	<code>ls file[123].txt</code> - Matches file1.txt, file2.txt, file3.txt
[! -]	Matches any single character not in the specified set.	<code>ls file[!12].txt</code> - Matches files that do not have 1 or 2 as the character after file
{ , }	Matches any of the patterns specified, separated by commas.	<code>ls file{*.py,*.txt}</code> - Matches Python and text files

Basic Regular Expression(BRE)

use ' ' (quats)!!

Pattern	Description	Example
.	Matches any single character. Similar to ? in globbing.	grep '.zip' usr-* - Matches lines containing zip preceded by any character
[]	Matches any single character within the brackets.	grep '[bc]zip' usr-* - Matches lines containing bzip or czip
^	Matches the beginning of a line.	grep '^[bm]zip' usr-* - Matches lines starting with bzip or mzip
\$	Matches the end of a line.	grep '[bm]zip\$' usr-* - Matches lines ending with bzip or mzip
[^]	Matches any single character not in the brackets. Similar to [!] in globbing.	grep '[^bm]zip' usr-* - Matches lines where zip is not preceded by b or m

Extended regular expressions(ERE)

use ' ' (quats)!!

Pattern	Description	Example
	Alternation, matches one of the specified patterns.	grep -E 'AAA BBB CCC'
()	Groups multiple elements into a single unit.	grep -E '^ (AAA BBB CCC)'
?	Matches zero or one occurrence of the preceding element.	grep -E '(abc)?' - Matches abc zero or one time
+	Matches one or more occurrences of the preceding element.	grep -E '(abc)+' - Matches abc one or more times
*	Matches zero or more occurrences of the preceding element.	grep -E '(abc)*' - Matches abc zero or more times
{x}	Matches exactly x occurrences of the preceding element.	grep -E '[0-9]{3}' - Matches exactly three digits
{x,y}	Matches between x and y occurrences of the preceding element.	grep -E '[0-9]{3,7}' - Matches between three and seven digits
{,y}	Matches up to y occurrences of the preceding element.	grep -E '[0-9]{,7}' - Matches up to seven digits
{x,}	Matches x or more occurrences of the preceding element.	grep -E '[0-9]{3,}' - Matches three or more digits
-o	Outputs only the matching parts of each line, rather than the whole line.	grep -o 'zip' usr-* - Only shows the matched zip part of each line

	Explanation	Users & groups
Whoami command	Displays the current user's username.	
Id command	Shows the user and group information, including UID and GID.	
/etc/passwd file	Contains user account information, with each user having a line entry: Username:Password:ID (UID):ID (GID)Info: directory: /shell.	
/etc/group file	Lists all groups and their members, with each group having a line entry: group_name:Password: ID (GID): List.	
/etc/shadow file	Stores encrypted user passwords and related password settings: Username : Password : Last password change : Minimum : Maximum : Warn : Inactive : Expire.	
SUDO	Allows a user to execute commands with the privileges of another user, typically the superuser.	Gid(27)
Purpose of Groups	Groups help manage permissions efficiently, allowing easier control over user access to files and resources.	
SUDO Group	Allows users to execute commands with elevated privileges, typically as the root user. This requires entering the user's password, cached for 15 minutes.	
Normal User	Regular users who can log in, access files, and perform standard operations. In Ubuntu, these users have a UID of 1000 or higher.	
System User	Users created for running background services and processes. These users typically cannot log in. In Ubuntu, they have a UID of 500 or less.	
Root User	The superuser with unrestricted access to the entire system. The root user has a UID of 0 and can perform any operation on the system. Uid = 0	
Linux Perspective	Linux provides flexibility and control over system appearance and behavior, unlike operating systems like Microsoft Windows, which may impose more restrictions.	
Shell Program	An interface for users to interact with the operating system via the terminal, executing commands and scripts.	

permission

chmod: Command to modify file and directory permissions.

1. `chmod <u,g,o> {+-=} <permission> <folder>`
2. `chmod <num,num,num>` (num= r, w, x)

Binary	Octal	String Representation
000	0 (0+0+0)	---
001	1 (0+0+1)	--x
010	2 (0+2+0)	-w-
011	3 (0+2+1)	-wx
100	4 (4+0+0)	r--
101	5 (4+0+1)	r-x
110	6 (4+2+0)	rw-
111	7 (4+2+1)	rwX

Special Permissions

- **Sticky Bit (t - 1777):** A special permission set on directories to restrict file deletion or renaming to the file owner, directory owner, or superuser. Used in directories like /tmp to protect user files from tampering.
- **SUID (s - 2777):** Set User ID permission for executable files. When a file has SUID, executing it allows a user to temporarily assume the file owner's privileges. Commonly used for system utilities like passwd.
- **SGID (s - 4777):** Set Group ID permission for files and directories. Files created within a directory with SGID inherit the group ownership of the directory. Useful for collaborative directories to ensure consistent group access.
- **Octal Deletion:** Remove special permissions using 0777.
- **Octal Full Permissions:** Full special permissions using 7777.

Visudo - sudoers

- **Visudo** - A command for edit the sudoers file safely. The sudoers file can configure permissions for specific users and commands.
- **Sudoers Syntax** -
user ALL=(user:group) NOPASSWD: <command> <file>
- User1 ALL=(administrator:administrator) NOPASSWD:
/usr/bin/nano /demo/file.
- **sudo -l** Lists sudo privileges for the current user.

```
GNU nano 2.9.3

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL

# Members of the admin group may su to any user
%admin   ALL=(ALL) ALL

# Allow members of group sudo to execute all commands
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more details

#includedir /etc/sudoers.d

```

^G Get Help ^O Write Out
^X Exit ^R Read File

/etc/shadow - structure

Field	Description
Username	The user's login name.
Password	The user's encrypted password.
Last Password Change	Days since the password was last changed.
Minimum Password Age	Minimum number of days required between password changes.
Maximum Password Age	Maximum number of days before a password must be changed.
Password Warning Period	Days before password expiration when the user is warned.
Password Inactivity Period	Days after password expiration when the account is disabled.
Account Expiry Date	Days since the epoch when the account will expire.
Reserved Field	Reserved for future use.

How To set an environmental variable?

1. Open the .bashrc or .bash_profile file in a text editor:

```
nano ~/.bashrc
```

2. Add the export command at the end of the file:

```
bash
```

```
export VARIABLE_NAME=value
```

3. Save the file and exit the editor.

```
source ~/.bashrc
```

CRONTAB

- **/var/spool/cron/crontabs:** Stores scheduled tasks for users; accessible only with sudo permissions.
- **crontab -l:** Lists the current user's scheduled cron jobs.
- **crontab -e:** Opens the cron table in the default editor for adding or editing scheduled tasks.
- **Cron Syntax:** Defines schedule with minute (0-59), hour (0-23), day of month (1-31), month (1-12), and day of week (0-6) fields.
- **Anacron:** If the computer was off and the commands did not run, anacron will run them as soon as the computer is turned on.
- **/etc/crontab:** Global cron table for system-wide scheduled tasks, often requiring root access to modify.

Package

- Packages contain binary files, pre and post-installation scripts, man pages, and configuration files, before installing any package, use the command “`sudo apt update`” to refresh the list of available packages.
- `sudo apt install <package name>`: Installs a specified package along with dependencies.
- `sudo apt remove <package name>`: Removes the binary files of a package but keeps configuration files.
- `sudo apt purge <package name>`: Completely removes a package, including configuration files.
- `sudo dpkg -i <file name>`: Installs a package from a downloaded file.
- `sudo dpkg -r <package name>`: Removes a package using dpkg.
- `sudo dpkg -l`: Lists installed packages and checks their status; “ii” indicates proper(correct) installation.

compression and archiving

- **Compression:** The process of reducing file size by removing redundancy. There are two types:
- **Lossless:** Preserves all data, allowing full restoration of the original file.
- **Lossy:** Removes some data, causing the restored file to differ from the original (e.g., JPEG, MP3).
- **gzip:** A tool used to compress files into .gz format, replacing the original file. The gunzip command decompresses these files back to their original state.
- **bzip2:** Uses the Burrows-Wheeler algorithm for better compression than gzip, resulting in .bz2 files. Use bunzip2 to decompress these files.
- **tar:** An archiving utility that combines multiple files into one .tar file. It supports options like c to create, x to extract, and t to list contents.
- **zip:** Both an archiving and compression tool that produces .zip files, often used for compatibility with Windows. Use unzip to extract files from a .zip archive.
- **Recursive Option:** A feature that allows compression tools to compress all files within a directory and its subdirectories (e.g., gzip -r).
- **Verbose Option:** Provides detailed output about the operations being performed by a command (e.g., gzip -v for compression details).

Bash scripting

What is Bash?

Explanation

Bash is a Unix shell and scripting language for task automation and system management.

Exit Status

Use exit 0 for success; check with **echo \$?** to see the last command's status.

Umask

Sets default file permissions:

Files: $666 - 022 = 644$, **Directories:** $777 - 022 = 755$

Variable Naming

Use different cases for readability:

Camel Case: firstName

Pascal Case: FirstName

Snake Case: first_name

Special Characters

Use \\$ or quotes to use \$ literally in scripts.

Creating a Script

Create with nano demo.sh, add **#!/bin/bash** at the top.

Concept	Code Example	Explanation
Variables	name="John" age=30	Define variables to store data.
Printing	echo "Hello, World!" echo "My name is \$name and I am \$age years old."	Use echo to print text and variable values to the terminal.
Input	read -p "enter your name: " first_name read -sp "enter your name: " first_name_secret	Read user input and store it in a variable.
Conditionals	if ["\$age" -ge 18]; then ... fi	Use if statements to execute code based on conditions.
Loops	for i in {1..5}; do echo "Number: \$i"; done	Use loops to repeat actions for a set of values or conditions.
Functions	greet() { echo "Hello, \$1!"; }	Define reusable code blocks with functions and pass arguments to them.
Brace Expansion	echo file{1..3}.txt	Create sequences of strings using brace expansion.
Grouping Commands	{ echo "Hello,"; echo "World!"; }	Group commands using curly braces to execute them together.
Tuples (Arrays)	tuple=("value1" "value2" "value3")	Use arrays to store multiple values in a single variable.
Accessing Tuple Elements	echo "First element: \${tuple[0]}"	Access elements in an array using index numbers.
Iterating Over a Tuple	for item in "\${tuple[@]"; do echo "\$item"; done	Loop through array elements with a for loop.
Length of Tuple	echo "Length of tuple: \${#tuple[@]}"	Determine the number of elements in an array.

Common scripts/commands

```
read -t 5 -p "Input your first name within 5 seconds: " name
read -n 2 -p "Input your age (max 2 digits): " age
read -s -N 5 -p "Enter your zip code (exactly 5 digits): " zipcode
echo "$name, $age, $zipcode" >> data.csv
```

```
readarray -t days < days.txt
```

```
readarray -t files < <(ls ~/Documents)
```

```
for file in "${files[@]}"; do
    if [ -f "$file" ]; then
        echo "$file already exists"
```

```
ping -c 4 -W 2 example.com
```

```
while read line; do
    commands...
done < file
```

```
while read line; do
    commands...
done < <(command)
```